

Heuristik-Matrix-OKR

Giriş ve Amaç

Bu projenin amacı, ağır kütüphaneler kullanmadan, tamamen temel matematik ve matris işlemlerine dayanan ilkel bir karakter tanıma ve şablon eşleştirme sistemi(görüntü işleme yapabilen ilkel bir yapay zeka) geliştirmektir. Projede, sisteme öğretilen örnek verilerin ortak özellikleri çıkarılmakta ve yeni girilen verilerin bu özelliklere ne kadar benzediği test edilmektedir.

Çalışma Mantığı

Sisteme gösterilen tüm örnek matrisler üst üste toplanır. Böylece harfin veya nesnenin en çok bastığı pikseller yüksek değerler olarak belirginleşir.

Toplam matristeki en küçük sayı bulunup tüm matristen çıkarılır. Amaç, her örnekte ortak olan gereksiz arka plan gürültülerini sıfırlamak ve sadece karakterin ana hatlarını öne çıkarmaktır.

Test edilen matris ile hafızadaki temiz şablon eleman elemana çarpılır ve sonuçlar toplanır. İki matris birbirine ne kadar benzerse, eşleşen piksellerin çarpımı o kadar büyük olacağından kümülatif skor da yüksek çıkar.

Temel Mantığın Kod Blokları İle Açıklanması

```
import numpy as np
toplam_matris = None
with open('matrisler.txt', 'r') as f:
    for satir in f:
        veri_matrisi = np.array([int(x) for x in satir.split()])
        if toplam_matris is None:
            toplam_matris = veri_matrisi
        else:
            toplam_matris += veri_matrisi
```

Burada önce boş bir toplam_matris oluşturulur ardından matrisler.txt dosyasından her satır alınır. Satırlardaki sayılar önce ayrılır, sonra veri_matrisine dönüştürülür ve toplanırlar.

```
temiz_toplam_matris = toplam_matris - np.min(toplam_matris)
print(temiz_toplam_matris)
```

Burada ise toplam matristeki en küçük sayı bulunup tüm matristen çıkarılır ve çıkan matris ekrana yazdırılır.

```
with open('matrisler.txt', 'r') as f:
    satir_no = 1
    for satir in f:
        sinav_matrisi = np.array([int(x) for x in satir.split()])
        skor = np.sum(sinav_matrisi * temiz_toplam_matris)
        print(f"Örnek {satir_no} Skoru: {skor}")
        satir_no += 1
```

Burada matrisler.txt dosyasındaki matrisleri sinav_matrisi adında, temiz_toplam_matris ile çarparak her birinin skoru hesaplanır. Bu skor geçicik bellek üzerinde tutulur, anlık olarak ekrana yazdırılır(skor karşılaştırması kullanıcıya bırakılmıştır). Bu işlemin yapılma nedeni, istenen şekil için en fazla alınabilecek puanları görmektir. Kullanıcının girdiği şekil ile makinenin öğrendiği şekil ne kadar benzerlerse puanı bu işlem sonucunda çıkanlara, öğrenilen şekillerin puanlarına o kadar yaklaşır.

```
kullanici_girdisi = input(f"Aralarında boşluk bırakarak matris girin:\n")
try:
    test_matrisi = np.array([int(x) for x in kullanici_girdisi.split()])
    if len(test_matrisi) != len(temiz_toplam_matris):
        print(f"Hata: Tam olarak {len(temiz_toplam_matris)} adet değer girmelisiniz!")
    else:
        yeni_skor = np.sum(test_matrisi * temiz_toplam_matris)
        print(f"Girdiğiniz matrisin skoru: {yeni_skor}")
except ValueError:
    print("Hata: Sadece sayı (0 veya 1) girmelisiniz!")
```

İlk yazılan bu kodun son bloğu olarak, kullanıcıdan karşılaştırma için matris istenir. İstenen matris matrisler.txt dosyasındaki matrisler uzunluğunda olması beklenir(matrisler.txt içindeki matrisler aynı uzunlukta olmalıdır). Kullanıcının girdiği matrisin de skoru hesaplanır ve çıktı olarak ekrana yazdırılır. Karşılaştırma bu ilk kod için kullanıcıya bırakılmıştır, kullanıcı skorları kendi karşılaştırır.

Örneğin; 3x3 ilk bir matris için olsun(istenildiği kadar büyütülebilir) ve şeklimiz — olsun, bu durumda matrisler.txt dosyası 110000, 011000 ... ihtimaller girilirdi(şekli oluşturacak şekilde).

Bu kodun skorları karşılaştırmama, görsel olarak betimleyememe gibi eksikleri vardır.

Kodun Geliştirilmesi

Isı Haritası Eklenmesi:

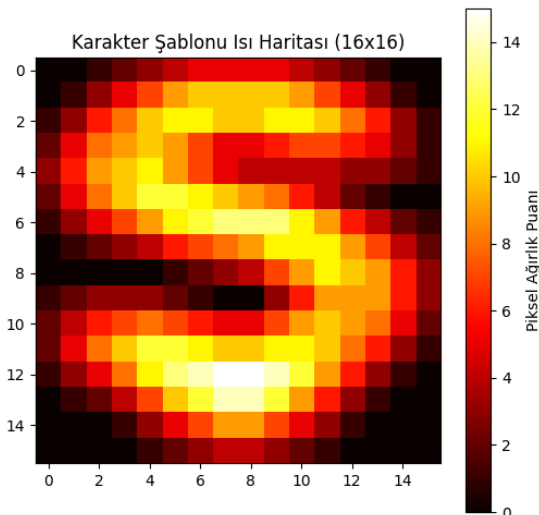
Görsel olarak temiz_toplam_matris'i görmemizi sağlamak için matplotlib kütüphanesini kullanacağız.

```
import matplotlib.pyplot as plt
```

Kütüphaneyi ekledikten sonra görseli oluşturulacak temiz_toplam_matris için kaç kaçlık matris olduğu belirlenmelidir. Ben kare matris kullandım, temiz_toplam_matris içindeki eleman sayısının kökünü alarak kaç kaç olduğu bulunur ve görsel buna uygun şekilde oluşturulur.

```
eleman_sayisi = len(temiz_toplam_matris)
boyut = int(np.sqrt(eleman_sayisi))
hafiza_resmi = temiz_toplam_matris.reshape((boyut, boyut))
plt.figure(figsize=(6, 6))
plt.title(f"Karakter Şablonu Isı Haritası ({boyut}x{boyut})")
plt.imshow(hafiza_resmi, cmap='hot')
plt.colorbar(label='Piksel Ağırlık Puanı')
plt.show()
```

Burada reshape ile hafiza_resmi gerçek bir matris yapılır. Buraya kadar elimizdeki matris gibi gösterdik ve öyleymiş gibi işlem yaptık. İmshow ile matris görselleştirilir ve cmap='hot' ile oluşturulan görseldeki renk dağılımları belirlenir.



Bu yanda görülen ısı haritası 16x16 belirlenmiş S karakterinin 15 olası halinin yukarıdaki işlemlerden geçerek temiz_toplam_matris haline gelmesiyle oluşmuştur.

Kullanıcıdan Görsel Girdi İstenmesi:

Baştaki kodda kullanıcı teker teker matrisi girmesi beklenirken daha görsel ve interaktif bir yöntem tercih etmek istedim.

```
import pygame
```

Burası için ise pygame kütüphanesi kullandım.

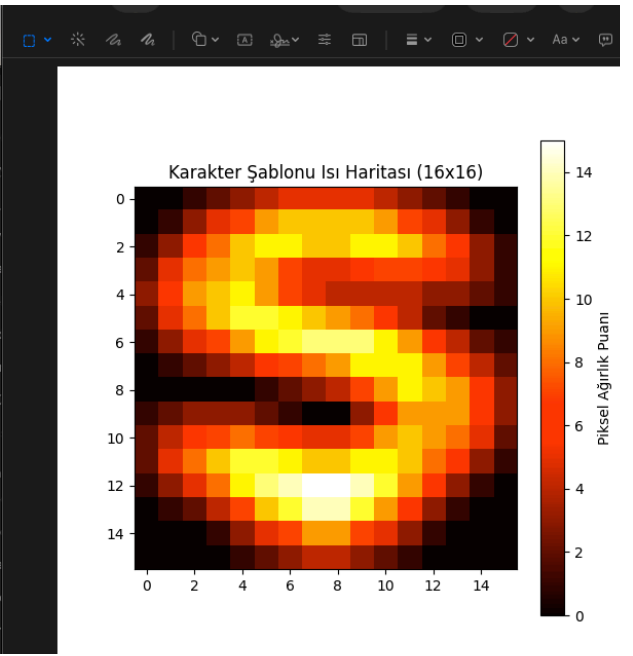
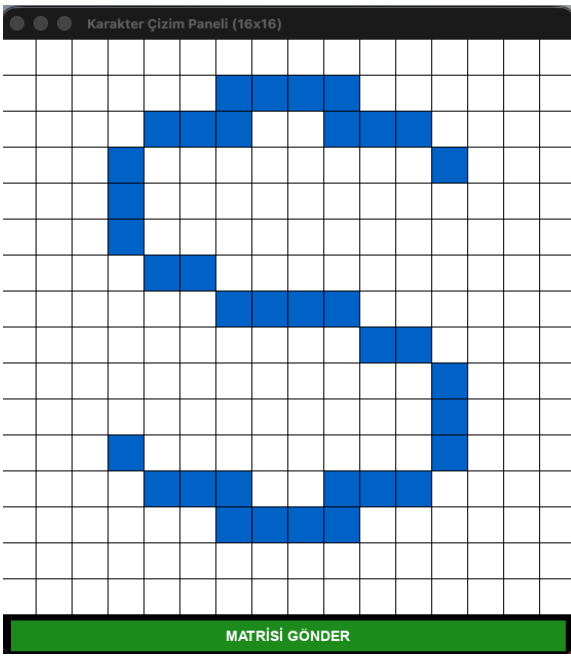
```
cizim_alani = np.zeros((boyut, boyut), dtype=int)
HÜCRE_BOYUTU = 35
PENCERE_BOYUTU = boyut * HÜCRE_BOYUTU
pygame.init()
pencere = pygame.display.set_mode((PENCERE_BOYUTU, PENCERE_BOYUTU + 40))#buton için +40
pygame.display.set_caption(f"Karakter Çizim Paneli ({boyut}x{boyut})")
font = pygame.font.SysFont("Arial", 14, bold=True)
calisiyor = True
while calisiyor:
    for r in range(boyut):
        for c in range(boyut):
            if çizim_alani[r, c] == 1 :
                renk = (0, 100, 200)#seçili kare renk
            else:
                renk = (255, 255, 255)
            pygame.draw.rect(pencere, renk, (c * HÜCRE_BOYUTU, r * HÜCRE_BOYUTU,
HÜCRE_BOYUTU - 1, HÜCRE_BOYUTU - 1))
        buton_alani = pygame.Rect(10, PENCERE_BOYUTU + 5, PENCERE_BOYUTU - 20, 30)
        pygame.draw.rect(pencere, (34, 139, 34), buton_alani)
        yazi = font.render("MATRİSİ GÖNDER", True, (255, 255, 255))
        pencere.blit(yazi, (PENCERE_BOYUTU//2 - yazi.get_width() // 2, PENCERE_BOYUTU + 12))
        pygame.display.flip()
    for olay in pygame.get_event_loop() if hasattr(pygame, 'get_event_loop') else
pygame.event.get():
        if olay.type == pygame.QUIT:
            calisiyor = False
            pygame.quit()
        elif olay.type == pygame.MOUSEBUTTONDOWN or (olay.type == pygame.MOUSEMOTION and
olay.buttons[0]):
            f_x, f_y = pygame.mouse.get_pos()
            if f_y < PENCERE_BOYUTU:
                c = f_x // HÜCRE_BOYUTU
                r = f_y // HÜCRE_BOYUTU
                if olay.type == pygame.MOUSEBUTTONDOWN:
                    çizim_alani[r, c] = 1 - çizim_alani[r, c]
                else:
                    çizim_alani[r, c] = 1
            elif buton_alani.collidepoint(f_x, f_y) and olay.type ==
pygame.MOUSEBUTTONDOWN:
                calisiyor = False
pygame.quit()
test_matrisi = çizim_alani.flatten()
yeni_skor = np.sum(test_matrisi * temiz_toplam_matris)
```

Öncelikle boş bir çizim alanı oluşturulur ve data type tam sayı olarak belirlenir. Hücre boyutu, pencere boyutu belirlenir ve pencere tanımlanır. Çizim alanı içerisindeki kareler, ızgaralar(küçük kareler) oluşturulur. Buton alanı ve buton oluşturulur.

Devamında gelen pygame.display.flip() kullanıcının seçtiği blokları görebilmesi için hayati bir önem taşıyor, while döngüsü altında olduğundan sürekli ekrana geri bildirim yaparak kullanıcıya ne yaptığını gösterir.

Olay kısmı için pygame.get_event_loop() kütüphanede var mı diye kontrol edilir yoksa pygame.event.get() kullanılır(sürüm desteği için). Kapatma tuşu ayarlanır ve mouse için girdiler belirlenir. Bir bloğa tekrar tıklanmasıyla durumunun değişmesi eklenir ve fare “Matrisi Gönder” yerine gelip tıklıyor mu diye kontrol eder.

En sonunda yeni skor hesaplanır.



- Örnek 1 Skoru: 746
Örnek 2 Skoru: 799
Örnek 3 Skoru: 805
Örnek 4 Skoru: 754
Örnek 5 Skoru: 664
Örnek 6 Skoru: 795
Örnek 7 Skoru: 887
Örnek 8 Skoru: 923
Örnek 9 Skoru: 887
Örnek 10 Skoru: 795
Örnek 11 Skoru: 664
Örnek 12 Skoru: 754
Örnek 13 Skoru: 805
Örnek 14 Skoru: 799
Örnek 15 Skoru: 746

Bu örnekte 15 farklı 16x16 matris içinde S şekli için skorlar ve benim çizdiğim S şekli için skor görülmektedir.

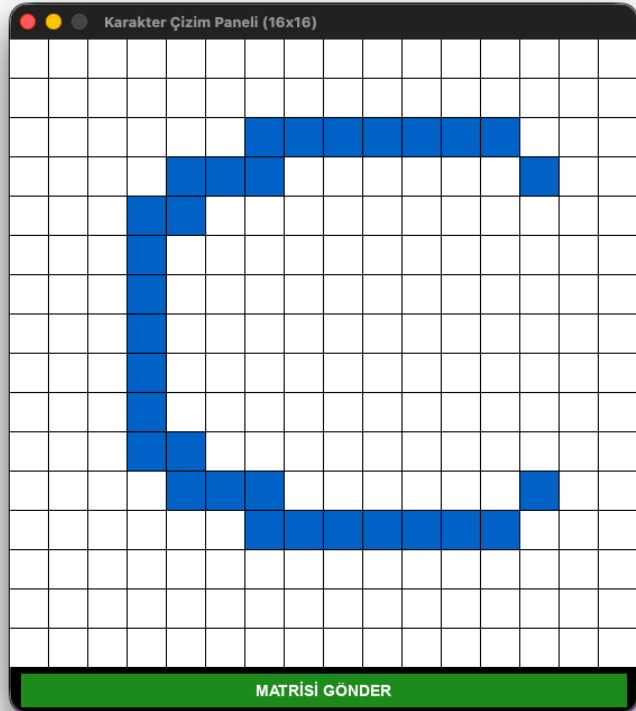
[ANALİZ SONUCU]

Çizdiğiniz matrisin benzerlik skoru: 375

Karşılaştırma adına C şekli çizilmiştir.
Benzerlik skoru: 287

Yani çizilen şekil bir S işaretine o kadar benzemiyor demek oluyor.

Ancak burada daha büyük bir hata göze çarpıyor eğer 8 çizecek olsaydık skor S çizdiğimizden daha yüksek olurdu.



Cezalandırma Sistemi:

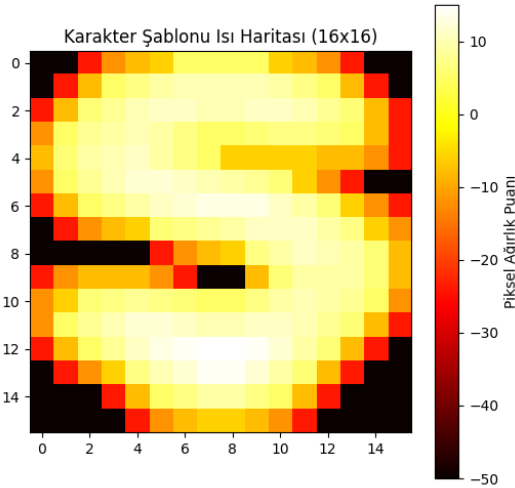
Buranın amacı biraz önce dediğim gibi S şeklini öğrenmeye çalışan makineye 8 çizseydik bunu S şekli sanacaktır. Bunun önüne geçmek için toplam matrisin boş kısımlarına ve çok denk gelmeyen yerler için (-) değerler veriyoruz.

Ben cezalandırma sistemide 0,1,2,3,4 değerlere sahip bloklar sahip oldukları değer ile ters orantılı (-) değer gelmesini sağlayacağım.

0 değeri için -50, 1 için $-24 \cdot (1/1)$, 2 için $-24 \cdot (1/2)$... şeklinde.

```
temiz_toplam_matris = toplam_matris.copy().astype(float)
it = np.nditer(temiz_toplam_matris, op_flags=['readwrite'])
for f in it:
    if f == 0:
        f[...] = -50
    elif 1 <= f <= 4:
        f[...] = -24 * (1 / f)
```

Görüldüğü gibi toplam matris temiz_toplam_matris olarak kopyalanır ve içinden teker teker elemanlar incelenip değiştirilir.



Yeni ısı haritası bu şekildedir.

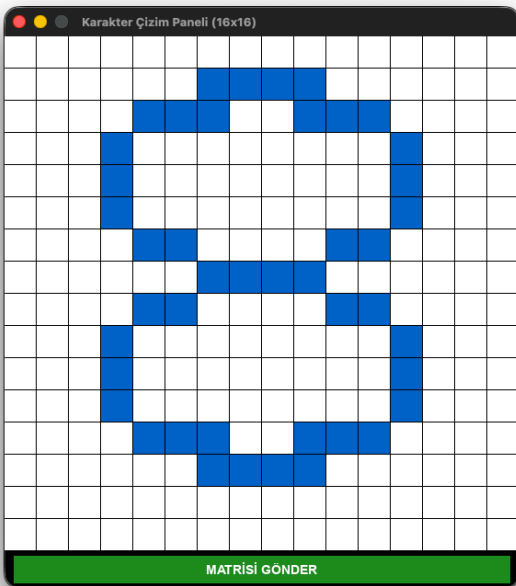
Yukarıda çizdiğim S şeklini tekrar çizmemle bu çizilen S şekli için skorun değişmediğini gördüm.

Çizdiğiniz matrisin benzerlik skoru: 375.0

Ancak yukarıda çizmiş olduğum C şeklini tekrar çizerek C şeklinin puanının düştüğünü gördüm.

Çizdiğiniz matrisin benzerlik skoru: 215.0

Son deneme olarak 8 işaretini yapalım.



Bu şekilde ise:

Çizdiğiniz matrisin benzerlik skoru: 300.0

Görüldüğü gibi ceza mekanizması güzel bir şekilde çalışıyor.

Birden Fazla Şekil Tanıma:

Tek bir S şekli ile sınırlı kalmak istemedim, karşılaştırmak adına daha fazla şekil ekleyebilmek için:

```
import sys
import glob
```

Glob kütüphanesi, dizinde bulunan tüm txt dosyalarını okumak için kullanılıyor.

```
dosya_listesi = glob.glob("*.txt")
if not dosya_listesi:
    print("Hata: Bulunulan klasörde hiç .txt dosyası bulunamadı!")
    sys.exit()
sablonlar = {}
boyut = None
print(f"Sistemde tespit edilen dosyalar: {dosya_listesi}")
for dosya in dosya_listesi:
```

Txt dosyalarını arar ve bulduklarını dosya_listesine ekler ve buradan teker teker eleman(dosya) çekerek döngüsünü başlatır.

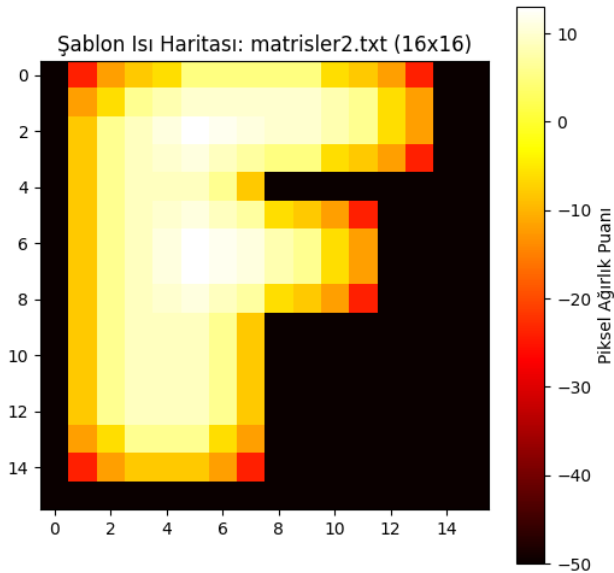
Bu döngünün içi önceden yazdığımız temiz_toplam_matris bulma ile aynıdır, bu nedenle buraya eklemedim.

```
sablonlar[dosya] = temiz_toplam_matris
```

Biraz daha ilerisinde, hala döngünün içindeyken temiz_toplam_matrisi kaydedilir. Devamında yine döngünün içinden ısı haritası çizilir.

```
for dosya_ismi, sablon_matrisi in sablonlar.items():
    yeni_skor = np.sum(test_matrisi * sablon_matrisi)
    print(f"Çizilen matrisin {dosya_ismi} şablonuna göre benzerlik skoru: {yeni_skor}")
```

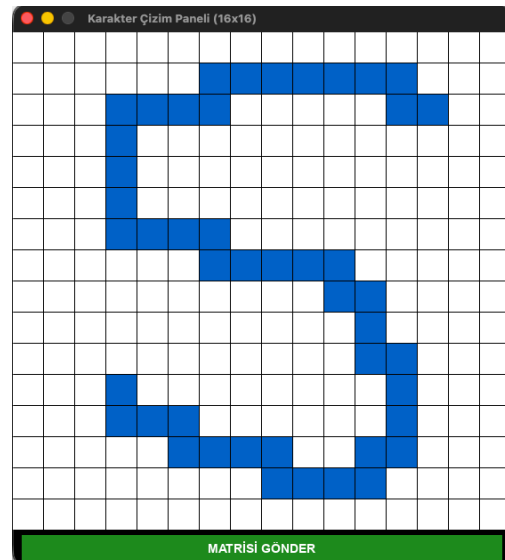
En sonda da bu şablonun içindekiler sablon_matrisi diye teker teker alınır ve skor hesaplanır. Şimdi matrisler2.txt adında yeni bir dosya açıp içine 15 farklı 16x16 F karakteri matrisi ekledim



Zamanımın kısıtlı oluşundan bu F karakteri için daha farklı ihtimallerin bulunduğu matrisler dosyası yapamadım.

Zaten S şekli için ısı haritası değişmedi bu nedenle buraya atmaya gerek duymadım.

İlk deneme için S şekline daha çok benzeyen bir şekil çizdim.



[matrisler2.txt SKOR ANALİZİ]

Örnek 1 Skoru: 182.0

Örnek 2 Skoru: 397.0

Örnek 3 Skoru: 438.0

Örnek 4 Skoru: 377.0

Örnek 5 Skoru: 191.0

Örnek 6 Skoru: 272.0

Örnek 7 Skoru: 497.0

Örnek 8 Skoru: 551.0

Örnek 9 Skoru: 467.0

Örnek 10 Skoru: 266.0

Örnek 11 Skoru: 219.0

Örnek 12 Skoru: 411.0

Örnek 13 Skoru: 426.0

Örnek 14 Skoru: 326.0

Örnek 15 Skoru: 90.0

[matrisler.txt SKOR ANALİZİ]

Örnek 1 Skoru: 504.0

Örnek 2 Skoru: 676.0

Örnek 3 Skoru: 690.0

Örnek 4 Skoru: 563.0

Örnek 5 Skoru: 298.0

Örnek 6 Skoru: 642.0

Örnek 7 Skoru: 877.0

Örnek 8 Skoru: 923.0

Örnek 9 Skoru: 866.0

Örnek 10 Skoru: 613.0

Örnek 11 Skoru: 247.0

Örnek 12 Skoru: 551.0

Örnek 13 Skoru: 679.0

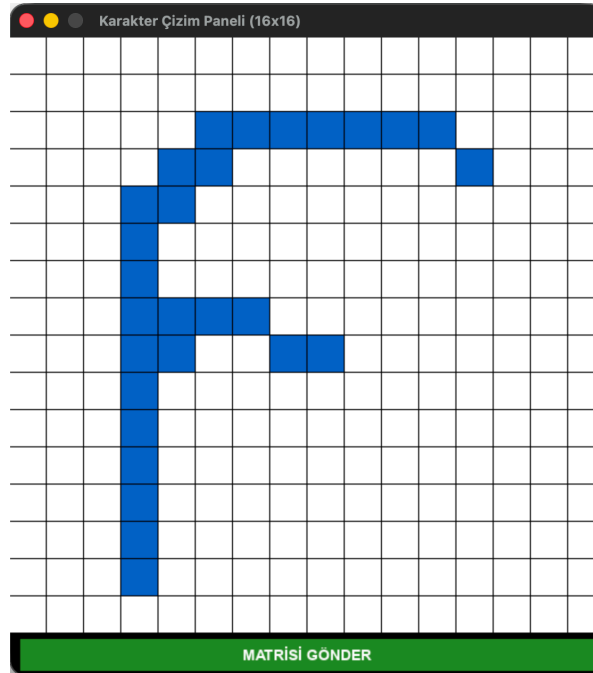
Örnek 14 Skoru: 654.0

Örnek 15 Skoru: 445.0

Çizdilen matrisin matrisler2.txt şablonuna göre benzerlik skoru: -434.0

Çizdilen matrisin matrisler.txt şablonuna göre benzerlik skoru: 394.0

Şimdi birde F şekline daha çok benzeyen bir şekil çizelim.



Çizdilen matrisin matrisler2.txt şablonuna göre benzerlik skoru: 214.0

Çizdilen matrisin matrisler.txt şablonuna göre benzerlik skoru: 9.0

Görüldüğü gibi her şey çalışıyor.

Bu projenin bütün kaynak koduna, matrisler.txt dosyalarına ve önceki sürümlerine GitHub hesabımdan erişebilirsiniz.

efemelis12@gmail.com

github.com/efeanilarlan